

Tokonomics

Active Claude token-optimization proxy + economics dashboard
Project Documentation

Local web app · Python stdlib server · MIT licensed

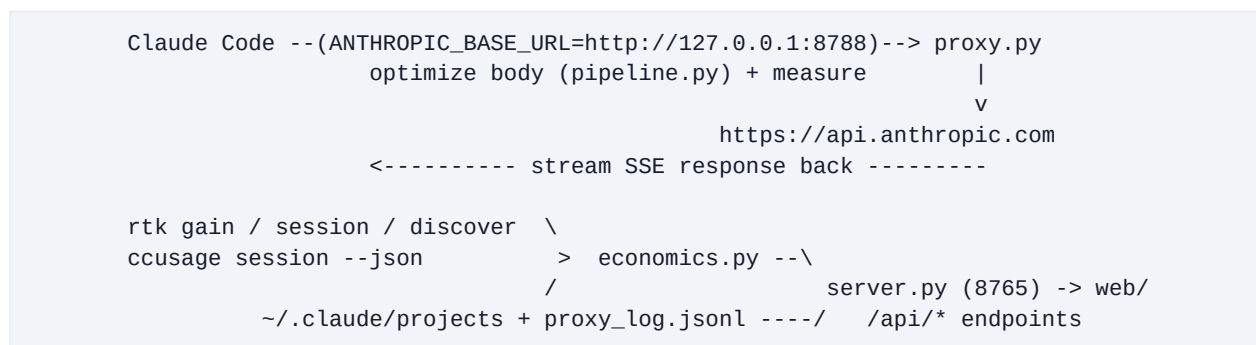
1. Overview

Tokonomics is a local web application with two halves that share one UI:

- **Active optimization proxy (main component)** — sits in the live request path between Claude Code and `api.anthropic.com`, runs each request through an optimization pipeline (rtk-style compression + markdown + prompt minify), and measures tokens saved before vs. after.
- **Economics dashboard** — reports historical spend, savings, per-session health, and where your tokens go, built on the external tools `rtk` and `ccusage`.

The dashboard half is a UI layer over external tools (never vendored). The proxy half is the active optimizer. Both run from one command and one local web server.

2. Architecture



No TLS interception is needed: Claude Code talks plain HTTP to the local proxy, and the proxy makes its own HTTPS call upstream, streaming the response straight back. Auth headers are forwarded untouched, so existing login keeps working.

Components

File	Responsibility
<code>tokonomics/proxy.py</code>	Streaming intercept proxy (port 8788). Forwards <code>/v1/*</code> , optimizes <code>/v1/messages</code> bodies, resolves exact token counts off the hot path, logs token-count-only records to <code>~/.tokonomics/proxy_log.jsonl</code> .
<code>tokonomics/pipeline.py</code>	The three optimization stages + a local token estimator (tiktoken or chars/4, fallback only). <code>optimize()</code> never raises.
<code>tokonomics/server.py</code>	Python stdlib HTTP server (dashboard + proxy control plane). Serves the web UI and the JSON APIs.
<code>tokonomics/economics.py</code>	Merges <code>rtk</code> + <code>ccusage</code> , computes the health score and day/week/month buckets, and the Insights breakdowns + recommendations.
<code>tokonomics/web/</code>	Vanilla JS + vendored Chart.js UI. Views: Overview, Optimize, Pulse, Insights.

3. The optimization proxy

Claude Code honours the `ANTHROPIC_BASE_URL` environment variable. Point it at the local proxy and traffic flows through Tokonomics on the way to Anthropic. For each `/v1/messages` request the proxy:

- reads and parses the JSON body;
- runs it through `pipeline.optimize()` (never raises; degrades to passthrough);
- forwards the optimized body upstream and streams the response back byte-for-byte (SSE or not);
- resolves exact token counts and appends a token-count-only log record — off the hot path.

4. The optimization pipeline

Three stages run over the request's message blocks. They only ever touch `tool_result` and document blocks (machine output: tool results, logs, files, RAG chunks) — never the system prompt or the user's own text.

Stage	What it does	Risk
rtk	Strips ANSI codes, collapses blank runs, dedupes identical lines in noisy command/log output. Truncation of huge outputs is opt-in.	Low
markdown	Converts base64 PDF/Office document blocks to clean markdown. Memoized by content hash.	Medium (doc fidelity)
prompt	Minifies embedded JSON only (whitespace there is insignificant). Leaves code / YAML / diffs untouched.	Low

Safety guarantees

- **Never raises into the live path** — any failure returns the original body (passthrough).
- **Deterministic + uniform** — a given tool result compresses to identical bytes whether it is the newest turn or deep in history. This keeps Anthropic prompt-caching prefixes byte-stable across turns; non-deterministic or position-dependent rewriting would force cache misses that cost more than they save.
- **Logs store token counts only** — never message content, never API keys.
- **Master OFF switch** turns the proxy into a pure relay (measure-only).

5. Exact token counting

Token counts shown to the user are **exact**, resolved off the hot path after the response streams:

- **After (optimized)** count comes free from the response usage (parsed from the `message_start` SSE event for streams).
- **Before (original)** count comes from a `/v1/messages/count_tokens` call.
- The local `tiktoken / chars-per-4` estimator is used only as a fallback when `count_tokens` is unavailable. Each request is tagged exact vs. estimate.
- `cache_read / cache_creation` are also recorded so the UI can flag if optimization is hurting prompt-cache hit rate.

6. The web views

Overview

Session health ring, Claude usage (sessions / tokens / spend), savings, untapped potential, a spend-vs-savings chart (day / week / month), top opportunities, and the per-session table.

Optimize

The headline control. A single master switch:

- **ON** — compresses tool output, logs, files and JSON before they reach Claude. Advanced per-stage toggles (rtk / markdown / prompt / truncate) are available below.
- **OFF** — measure-only: requests reach Claude unchanged, Tokonomics just counts tokens and money. The advanced toggles grey out.

Also shows the connect command, original vs. optimized tokens, % and \$ saved, request count, and exact-vs-estimate + cache health.

Pulse

A live feed of recent proxied requests (time, model, original → optimized, saved, status) — token counts only.

Insights

Where your tokens go, with an All / per-project dropdown:

- **Tokens by project** — per-project token totals and cost.
- **Most-read files** — top files read/edited, with count and share (re-reads re-upload every turn).
- **Most-run commands** — top bash commands grouped (e.g. `git add`), with count and percentage.
- **Recommendations** — actionable advice derived from the data (noisy commands, heavy re-reads, proxy off, low cache hit, spend concentration).

This is read-only analysis of local `~/claude/projects` transcripts joined with ccusage totals — the proxy logs nothing extra.

7. Health score

Health blends prompt quality with how much the proxy is actually optimizing:

$$\text{health} = 0.5 \times \text{quality (cache reuse)} + 0.5 \times \text{optimization (proxy savings \%)}$$

- **quality** = $\text{cacheRead} / (\text{cacheRead} + \text{cacheCreate} + \text{input})$ — how efficiently context is reused (from ccusage).
- **optimization** = the live proxy's measured savings rate (0 when the proxy is off or idle).

The score rises as the proxy saves tokens on real traffic — a metric you control, replacing the earlier rtk-adoption input.

8. Setup & run

```
pip install -e .[proxy]          # optional: enables markdown + tiktoken
python -m tokonomics             # dashboard http://127.0.0.1:8765 + proxy :8788
python -m tokonomics --port 9000 --proxy-port 9001 --price 3.5
python -m tokonomics --no-proxy # dashboard only
```

Route Claude Code through the proxy in a new shell, then start c laude:

```
setx ANTHROPIC_BASE_URL "http://127.0.0.1:8788"      :: detach later: setx ANTHROPIC_BASE_URL ""
```

Prerequisites: Python 3.9+. Dashboard data needs rtk (scripts/install-rtk) and ccusage (npm i -g ccusage). The proxy's full pipeline needs `pip install -e .[proxy]`.

Auto-start on login (Windows)

- `scripts/start-tokonomics.bat` and `.ps1` launch the app minimized and log to `%TEMP%\tokonomics.log`.
- `Tokonomics.bat` in the Startup folder auto-runs on login. (Windows does not auto-run `.ps1` from Startup — the `.bat` is the effective entry.)

9. API endpoints

Endpoint	Purpose
GET /api/economics	Overview payload: health, totals, buckets, sessions, opportunities.
GET /api/insights	Per-project tokens, most-read files, most-run commands, recommendations.
GET /api/config	rtk/ccusage presence, price.
GET /api/proxy/status	Running?, port, upstream, pipeline config, counters.
GET /api/proxy/stats	Aggregated savings, per-stage, cache health, recent requests.
GET /api/proxy/setup	Exact ANTHROPIC_BASE_URL set/unset commands.
POST /api/proxy/{start,stop,config}	Start/stop the proxy; toggle pipeline stages / passthrough.

10. Privacy, conventions & license

- **Privacy** — the proxy handles real API traffic but logs token counts only; never message content, never API keys. Insights reads local transcripts already on disk.
- **Dependencies (scoped)** — the dashboard server is stdlib-only; the UI vendors Chart.js (MIT); the optional proxy may use markdown + tiktoken (lazily imported, so the app runs without them).
- **External tools** — rtk and ccusage are invoked as programs, never vendored. Attribution lives in `THIRD_PARTY_NOTICES.md`.

- **License** — MIT.

Generated from docs/build_docs.py · Tokonomics is not affiliated with or endorsed by the authors of rtk or ccusage.